

Report

Task1-1

在 nix-vector-helper.h 中加入 SetPathFile()在 public

```
*/ the routing path at the specified time.
*/
void PrintRoutingPathAt (Time printTime, Ptr<Node> source, IpAddress dest)
void SetPathFile(const std::string& pathFile); //task1-1
```

在 private 中加入變數 m_pathFile

```
static void PrintRoute (Ptr<Node> source)
std::string m_pathFile; //task1-1
```

在 nix-vector-helper.cc 中的 NixVectorHelper 加入 m_pathfile

```
template <typename T>
NixVectorHelper<T>::NixVectorHelper (const NixVectorHelper<T> &o)
: m_agentFactory (o.m_agentFactory),
  m_pathFile(o.m_pathFile) //task1-1
{
}
```

把外部的 pathfile 存到內部變數 m_pathfile 中讓其他 function 用

```
template <typename T>
void
NixVectorHelper<T>::SetPathFile(const std::string& pathFile) //task1-1
{
  m_pathFile = pathFile;
}
```

Call SetPaths in Create function

```
NixVectorHelper<T>::Create (Ptr<Node> node) const
{
  Ptr<NixVectorRouting<IpRoutingProtocol>> agent = m_
  agent->SetNode (node);
  node->AggregateObject (agent);
  if(!m_pathFile.empty()){ // task1-1
    agent->SetPaths(m_pathFile);
  }
  return agent;
}
```

在 nix-vector-routing.h 的 public 中加入 setpaths

```
void PrintRoutingPath (Ptr<Node> source, IpAd  
// task1-1  
void SetPaths(const std::string &pathFile);
```

在 private 中加入 m_pathfile 跟 m_table 用起點終點當 key 內容存路徑序列

(用 map 存)

```
// task1-1  
std::string m_pathFile; //從Helper傳進來的pathFile路徑  
std::map<std::pair<uint32_t,uint32_t>, std::vector<uint32_t>> m_table; // key=(srcId,dstId) value=完整節點序列
```

在 nix-vector-routing.cc 中加入 function SetPaths，先把要開的 file 打開，每一行前面三個分別代表起點終點跟路徑長度，之後一個一個 node 讀，並檢查讀進來的 node 格式有沒有正確，正確再存到路徑中，如果讀到的數量跟開始存的總路徑長一樣，代表有完整讀到，就存入前面宣告的 table 中。

```
template <typename T>  
void  
NixVectorRouting<T>::SetPaths(const std::string &pathFile)  
{  
    m_pathFile = pathFile; //把pathFile存下來  
    std::ifstream infile(pathFile); //開檔案  
    uint32_t srcId, dstId, pathLen; //起點終點路徑長度  
    while(infile >> srcId >> dstId >> pathLen){  
        std::vector<uint32_t> path;  
        path.reserve(pathLen); //用來存路徑大小開獨到的路徑長  
        uint32_t nodeId;  
        bool formatOk = true; //用來檢查讀進來的東西是否格式正確  
        for(uint32_t i = 0; i < pathLen; ++i){  
            if(!(infile >> nodeId)){  
                NS_LOG_ERROR ("NixVectorRouting::SetPaths: format error " << i << " ID");  
                formatOk = false;  
                break;  
            }  
            path.push_back(nodeId); //對的話存路徑  
        }  
        if (formatOk && path.size() == pathLen) {  
            m_table[{srcId, dstId}] = path;  
        }  
    }  
    infile.close();  
}
```

Task1-2

在 nix-vector-routing.cc 中的 GetNixVector 加入用自己的方法去 BuildNixVector 的 code

如果 `m_pathfile` 不是空代表要使用我們傳進去檔案的 `path`，先找出起點與終點，用起點終點當 `key` 去找出在 `table` 中的位置，取出前面存的路徑序列，接著設置 `parentVector`，起點的 `parent` 設為自己，之後每一個點的 `parent` 設為前一跳的點，即從 36 到 5，5 的 `parent` 會設置為 36，用自己建立好的 `parentVector` 去 `BuildNixVector()`。如果 `m_pathfile` 是空的話就會執行原本的 BFS 部分。

```
// Task1-2
if (!m_pathFile.empty())
{
    //找出SD pair在table中存的路徑
    auto key = std::make_pair(source->GetId(), destNode->GetId());
    auto it = m_table.find(key);
    if (it != m_table.end()){
        const std::vector<uint32_t>& hops = it->second; //hops為路徑序列
        uint32_t nNodes = NodeList::GetNNodes(); //有幾個node
        //用來存每一個node的 parent
        std::vector<Ptr<Node>> parentVector (nNodes, Ptr<Node> ());
        //source的parent設為自己
        parentVector[source->GetId()] = source;
        for(size_t i = 1; i < hops.size (); ++i){
            uint32_t currId = hops[i]; //hop[i]的parent=hops[i-1]
            uint32_t prevId = hops[i - 1];
            parentVector[currId] = NodeList::GetNode(prevId);
        } //用parentVector建立NixVector
        if(BuildNixVector (parentVector,source->GetId(),destNode->GetId(),nixVector)){
            return nixVector;
        }
    }
}
```

Task1-3

我直接把 `GetNixVectorInCache()` and `GetIpRouteInCache()` 的 code 都移除，改成一律都用 `GetNixVector()`。先處理 `total neighbor`，如果是 0 代表還沒有計算，先計算。算完後用 `total neighbor` 算要讀幾位元並提取出下一跳的 `index`

```
//直接移除GetNixVectorInCache() and GetIpRouteInCache()
nixVector = GetNixVector(m_node, destAddress, oif); //改成一律call GetNixVector()
if(nixVector)
{
    NS_LOG_LOGIC ("Nix-vector contnets: " << *nixVector);
    nixVectorForPacket = nixVector->Copy(); //複製一份給packet
    if (m_totalNeighbors == 0) //算total neighbors
    {
        m_totalNeighbors = FindTotalNeighbors (m_node);
    }
    uint32_t nBits = nixVectorForPacket->BitCount (m_totalNeighbors); //要讀取多少位元
    uint32_t nodeIndex = nixVectorForPacket->ExtractNeighborIndex (nBits); //下一跳的index
    IpAddress gatewayIp; //由index找出gatewayIp與裝置index
    uint32_t devIndex = FindNetDeviceForNixIndex (m_node, nodeIndex, gatewayIp);
```

決定介面，如果有指定輸出裝置 `oif` 就使用，否則用剛剛找出的 `devIndex` 去找介面，選出要使用的 `source ip`，建立一個 `IpRoute` 並把 `source ip`, `destination ip` 等資訊放入並送出。最後加入前面算出的 `nixVector(nixVectorForPacket)`到封包中用來當下層轉送傳輸的資訊。

```
//決定要用哪個介面
int32_t iface = oif ? m_ip->GetInterfaceForDevice (oif)
                : m_ip->GetInterfaceForDevice (m_node->GetDevice (devIndex));
NS_ASSERT_MSG (iface >= 0, "Unable to find corresponding interface");
//選出source ip
IpAddress sourceIp = m_ip->SourceAddressSelection (iface, destAddress);

rtentry = Create<IpRoute> (); //建立新的IpRoute
rtentry->SetSource      (sourceIp);
rtentry->SetDestination (destAddress);
rtentry->SetGateway      (gatewayIp);
rtentry->SetOutputDevice(
    oif ? oif : m_ip->GetNetDevice (iface)
);
sockerr = Socket::ERROR_NOTERROR;

if (p) //把NixVector附到packet上
{
    NS_LOG_LOGIC ("Attaching Nix-Vector to packet: " << *nixVectorForPacket);
    p->SetNixVector (nixVectorForPacket);
}
```

Task2-1

改成 `return Ptr<PacketSink>`因 `task3` 會用到，要算 `throughput`。先取出起點跟終點 `node id`，與終點的 `ip address`，要用 `GetAddress (1, 0)`才會是對的，`(0,0)`會導致自己 `ip` 傳給自己 `ip` 導致收不到封包，在終點裝 `PacketSink` 用來收 `tcp` 流量，起點裝 `bulkSender` 用來傳輸封包(與 `lab4` 相同)，其中 `maxBytes` 我在 `call sendpacket` 的時候會自己帶進來，因 `task2,3` 的要求不同。

```
Ptr<PacketSink>
SendPacket (int srcId, int dstId, uint32_t maxBytes){
    // Task 2.1: Complete this function
    // Task 2.1: Set MaxBytes to 512 & Task 3.1: Set MaxBytes to 0
    Ptr<Node> src = NodeList::GetNode (srcId); //取出起點終點的node
    Ptr<Node> dst = NodeList::GetNode (dstId);
    Ptr<Ipv4> ipv4 = dst->GetObject<Ipv4> (); //取終點的ip address
    Ipv4Address dstAddr = ipv4->GetAddress (1, 0).GetLocal ();
    //在終點裝PacketSink用來接收tcp流量
    PacketSinkHelper sinkHelper("ns3::TcpSocketFactory", InetSocketAddress (Ipv4Address::GetAny (), port));
    ApplicationContainer sinkApps = sinkHelper.Install(dst);
    sinkApps.Start (Seconds (0.0));
    Ptr<PacketSink> sink = DynamicCast<PacketSink>(sinkApps.Get(0));
    //max=512, sendsize=512 裝BulkSendApplication到起點src上用來做tcp傳送
    BulkSendHelper bulkSender ("ns3::TcpSocketFactory", InetSocketAddress (dstAddr, port));
    bulkSender.SetAttribute ("MaxBytes", UintegerValue (maxBytes)); //task2,3修改maxbyte
    bulkSender.SetAttribute ("SendSize", UintegerValue (512));
    ApplicationContainer srcApps = bulkSender.Install(src);
    srcApps.Start (Seconds (0.0));
    return sink;
}
```

Task2-2, task3-2

我直接把三組存成一個 vector，之後看 task 是 2 或 3 去決定要把幾組去 call sendpacket。先建立一個 vector<Ptr<PacketSink>>，要將 sendpacket 的結果存入在後面算 throughput，如果是 task2 就傳第一組就好，並把 maxbytes 設為 512，如果是 task3，要把三組都 call sendpacket，並把 maxbytes 設為 0。

```
// Task 2.2 & Task 3.2 : Call SendPacket()
vector<pair<int,int>> flows = {{36,38},{37,40},{39,41}};
```

```
vector<Ptr<PacketSink>> sinks;
sinks.reserve(flows.size());
if(Task == 2) {
    Simulator::Schedule(Seconds(1e-9), &connect);
    sinks.push_back(SendPacket(flows[0].first, flows[0].second, 512));
}
else {
    for (auto &f : flows) {
        sinks.push_back(SendPacket(f.first, f.second, 0));
    }
}
```

Task2-3

判斷封包用 size 來看，要大於 150 以上才會印出來，用來過濾太小的。
Context 中的文字會等於註解的那段，拿出 node ID 要從 NodeList 與 DeviceList，並看是 Tx 還是 Rx，最後輸出整段資訊。

```
static void
EchoMacTxRx (std::string context, const Ptr<const Packet> packet)
{
    uint32_t sz = packet->GetSize ();
    //判斷方法直接看size 把小的都過濾掉
    if (sz < 150) return;
    //context會等於/NodeList/36/DeviceList/0/$ns3::MockNetDevice/MacRx
    //要拿出nodeID從NodeList與DeviceList中間拿
    size_t off = context.find("/NodeList/") + 10;
    size_t end = context.find("/DeviceList", off);
    uint32_t nodeId = std::stoul(context.substr(off, end - off));
    //看是Tx還是Rx
    bool isTx = (context.find("MacTx") != std::string::npos);
    double t_ns = Now().GetNanoSeconds ();
    std::cout << (isTx ? "MacTx" : "MacRx") << " at node: " << nodeId
    << ", now=+" << std::scientific << t_ns << " ns" << std::endl;
}
```

Task2 result

```
MacTx at node: 36, now=+1.175357e+09 ns
MacRx at node: 0, now=+1.665179e+09 ns
MacTx at node: 0, now=+1.665179e+09 ns
MacRx at node: 5, now=+1.834476e+09 ns
MacTx at node: 5, now=+1.834476e+09 ns
MacRx at node: 35, now=+2.003400e+09 ns
MacTx at node: 35, now=+2.003400e+09 ns
MacRx at node: 3, now=+2.174515e+09 ns
MacTx at node: 3, now=+2.174515e+09 ns
MacRx at node: 38, now=+2.178520e+09 ns
```

Task3-1 在傳送時把 maxbytes 改成 0 就好，在上面的 task2-2,3-2 中有提到

Task3-2 把三組都 call sendpacket 就好。

Task3-3

判斷如果 task 等於 3 時，代表要算 throughput，把前面存 Ptr<PacketSink>的 vector sink 拿出來，對每一組用 GetTotalRx()來看總傳輸的 throughput，並印出來，最後把全部的量加起來得到 total throughput。

```
if(Task == 3) {
    uint64_t totalBytes = 0;
    for(size_t i = 0; i < flows.size(); ++i) {
        uint64_t bytesRx = sinks[i]->GetTotalRx();
        std::cout << flows[i].first << "->" << flows[i].second << ": " << bytesRx << std::endl;
        totalBytes += bytesRx;
    }
    std::cout << "Total throughput: " << totalBytes << std::endl;
}
```

Task3 result

Path1

```
36->38: 38400
37->40: 37376
39->41: 10752
Total throughput: 86528
```

Path2

```
36->38: 95232
37->40: 95744
39->41: 118272
Total throughput: 309248
```

Questions

1.

我的作法是先建立 `parentVector(nNodes, Ptr<Node>())`，大小是總路徑長度，初始都指向 `null`，把 `source` 的 `parent` 設為自己，後面的點的 `parent` 都指向前一跳的點，如此形成一個完整路徑，之後 `build` 的時候會從中點開始，用 `parent` 去推出前一個點是誰，以此類推不斷往前推，最後在反向就可以得到從起點到終點的完整路徑

```
(BuildNixVector (parentVector,source->GetId(),destNode->GetId(),nixVector))
```

2.

Congestion 在 `path1` 中有發生，因為三組 SD pair 的路徑中都有經過 `node35` 這點，而三組同時共享這個點會造成 congestion，導致 throughput 降低，且 `(36,38)`與`(39,41)`路徑又都有 `node5`，`(37,40)`與`(39,41)`路徑也都有 `node35`，都會使 throughput 降更低，其中`(39,41)`因重疊共享的 node 最多，導致 throughput 非常低。在 `path2` 中，三組 SD pair 的路徑完全沒有重複，代表每組都可以使用完整的流量，不用共享，所以 throughput 很大，其中`(39,41)`的路徑最短，所以 throughput 是三組中最大的。

3.

我把 `path1` 中改成只有`(36,38)`這組路徑，其他兩組沒有時 throughput 如下圖，可以發現只有這組時跟 `path2` 的結果差不多，代表與沒有 congestion 時的結果一樣，所以原本三組路徑都有的情況會有 congestion。

```
36->38: 95232
37->40: 0
39->41: 0
Total throughput: 95232
```

再來我改成有`(36,38)`與`(37,40)`這兩組路徑，這兩組有共用一個 `node35`，結果如下圖，可以很明顯看出有 congestion 發生使`(36,38)`的 throughput 降低。

```
36->38: 1536
37->40: 84480
39->41: 0
Total throughput: 86016
```

最後我只跑(39,41)這組，結果如下圖，可以發現比起原本大了很多，代表沒有 congestion，使 throughput 變很大，原本有 congestion 才使 throughput 很小。

```
36->38: 0
37->40: 0
39->41: 95232
Total throughput: 95232
```

Initial commit 我只更改了 studentID

Second commit 我把 code 的部分都完成

Final commit 我把 report 寫完